04_bronze_to_silver_fix (Python)

Import Notebook

```
%md
# 04 — Bronze to Silver (Databricks DQX Profiling + Cleaning + Quarantine)
- Installs and uses the **Databricks Labs DQX** library for data profiling and quality validation
- Profiles each Bronze table using `DQProfiler`
- Generates quality rule candidates using `DQGenerator`
- Applies quality checks using `DQEngine` with `apply_checks_and_split`
- Splits into valid (→ Silver) and quarantined (→ Quarantine)
- Cleans/standardizes valid records using DataFrame transformations
- Logs DQ results to metadata
```

04 — Bronze to Silver (Databricks DQX Profiling + Cleaning + Quarantine)

- Installs and uses the **Databricks Labs DQX** library for data profiling and quality validation
- Profiles each Bronze table using `DQProfiler`
- Generates quality rule candidates using `DQGenerator`
- Applies quality checks using `DQEngine` with `apply_checks_and_split`
- Splits into valid (→ Silver) and quarantined (→ Quarantine)
- Cleans/standardizes valid records using DataFrame transformations
- Logs DQ results to metadata

```
%pip install databricks-labs-dqx
```

```
Collecting databricks-labs-dqx Downloading databricks_labs_dqx-0.13.0-py3-none-any.whl.metadata (4.2 kB) Collecting databricks-labs-blueprint<0.10,>=0.9.1 (from databricks-labs-dqx) Downloading databricks_labs_blueprint-0.9.3-py3-none-any.whl.metadata (55 kB) Collecting databricks-sdk~=0.73 (from databricks-labs-dqx) Downloading databricks_sdk-0.102.0-py3-none-any.whl.metadata (40 kB) Collecting pyyaml~=6.0.3 (from databricks-labs-dqx) Downloading pyyaml-6.0.3-cp311-cp311-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (2.4 kB) Collecting sqlalchemy<3.0,>=2.0 (from databricks-labs-dqx) Downloading sqlalchemy-2.0.48-cp311-cp311-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (9.5 kB) Requirement already satisfied: requests<3,>=2.28.1 in /databricks/python3/lib/python3.11/site-packages (from databricks-sdk~=0.73->databricks-labs-dqx) (2.31.0) Requirement already satisfied: google-auth~=2.0 in /databricks/python3/lib/python3.11/site-packages (from databricks-sdk~=0.73->databricks-labs-dqx) (2.35.0) Collecting protobuf!=5.26.*,!=5.27.*,!=5.28.*,!=5.29.0,!=5.29.1,!=5.29.2,!=5.29.3,!=5.29.4,!=6.30.0,!=6.30.1,!=6.31.0,<7.0,>=4.25.8 (from databricks-sdk~=0.73->databricks-labs-dqx) Downloading protobuf-6.33.6-cp39-abi3-manylinux2014_aarch64.whl.metadata (593 bytes) Collecting greenlet>=1 (from sqlalchemy<3.0,>=2.0->databricks-labs-dqx) Downloading greenlet-3.3.2-cp311-cp311-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl.metadata (3.7 kB) Requirement already satisfied: typing-extensions>=4.6.0 in /databricks/python3/lib/python3.11/site-packages (from sqlalchemy<3.0,>=2.0->databricks-labs-dqx) (4.10.0) Requirement already satisfied: cachetools<6.0,>=2.0.0 in /databricks/python3/lib/python3.11/site-packages (from google-auth~=2.0->databricks-sdk~=0.73->databricks-labs-dqx) (5.5.0) Requirement already satisfied: pyasn1-modules>=0.2.1 in /databricks/python3/lib/python3.11/site-packages (from google-auth~=2.0->databricks-sdk~=0.73->databricks-labs-dqx) (0.2.8) Requirement already satisfied: rsa<5,>=3.1.4 in /databricks/python3/lib/python3.11/site-packages (from google-auth~=2.0->databricks-sdk~=0.73->databricks-labs-dqx) (4.9) Requirement already satisfied: charset-normalizer<4,>=2 in /databricks/python3/lib/python3.11/site-packages (from requests<3,>=2.28.1->databricks-sdk~=0.73->databricks-labs-dqx) (2.0.4) Requirement already satisfied: idna<4,>=2.5 in /databricks/python3/lib/python3.11/site-packages (from requests<3,>=2.28.1->databricks-sdk~=0.73->databricks-labs-dqx) (3.4) Requirement already satisfied: urllib3<3,>=1.21.1 in /databricks/python3/lib/python3.11/site-packages (from requests<3,>=2.28.1->databricks-sdk~=0.73->databricks-labs-dqx) (1.26.16) Requirement already satisfied: certifi>=2017.4.17 in
```

```
dbutils.library.restartPython()
```

```
# Widget parameters
dbutils.widgets.text("catalog_name", "workspace", "Catalog Name")
dbutils.widgets.text("schema_name", "raw_zone", "Schema Name")
dbutils.widgets.text("base_path", "/Volumes/workspace/raw_zone/chinook", "Base Path")
dbutils.widgets.text("table_name", "all", "Table Name")
dbutils.widgets.text("run_id", "", "Run ID")

catalog = dbutils.widgets.get("catalog_name")
run_id_param = dbutils.widgets.get("run_id")
```

```
from pyspark.sql.functions import (
    col, trim, lower, upper, when, lit, current_timestamp,
    count, explode
```

```
)
from pyspark.sql.types import StringType, DecimalType
from databricks.labs.dqx.profiler.profiler import DQProfiler
from databricks.labs.dqx.profiler.generator import DQGenerator
from databricks.labs.dqx.engine import DQEngine
from databricks.labs.dqx.rule import DQRowRule
from databricks.labs.dqx import check_funcs
from databricks.sdk import WorkspaceClient
from datetime import datetime
import uuid
import yaml

try:
    run_id = dbutils.jobs.taskValues.get(taskKey="task_03_raw_to_bronze", key="run_id")
except:
    run_id = run_id_param if run_id_param else str(uuid.uuid4())[:8]

ws = WorkspaceClient()
dq_engine = DQEngine(ws)
profiler = DQProfiler(ws)
generator = DQGenerator(ws)

print(f"Run ID: {run_id}")
print("Databricks DQX engine initialized successfully")
```

```
Run ID: 2d0ac997 Databricks DQX engine initialized successfully
```

```
%md
## 1. Helper: Cast Decimal Columns to Double for DQX Compatibility
DQX profiler cannot handle Decimal types — cast them to Double before profiling.
```

1. Helper: Cast Decimal Columns to Double for DQX Compatibility

DQX profiler cannot handle Decimal types — cast them to Double before profiling.

```
def cast_decimals_to_double(df):
    """Cast all Decimal columns to Double for DQX profiler compatibility."""
    for field in df.schema.fields:
        if isinstance(field.dataType, DecimalType):
            df = df.withColumn(field.name, col(field.name).cast("double"))
    return df
```

```
%md
## 2. DQX Rules Definition Per Table
Using `DQRowRule` from `databricks.labs.dqx.rule` with built-in check functions.
Note: `is_in_range` requires both `min_limit` and `max_limit`.
```

2. DQX Rules Definition Per Table

Using `DQRowRule` from `databricks.labs.dqx.rule` with built-in check functions. Note: `is_in_range` requires both `min_limit` and `max_limit`.

```
def get_dqx_rules(table_name):
    """Return DQX rules for each Chinook table using DQRowRule objects."""
    rules = {
        "customer": [
            DQRowRule(name="not_null_customerid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="CustomerId"),
            DQRowRule(name="not_null_email", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="Email"),
            DQRowRule(name="not_null_firstname", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="FirstName"),
            DQRowRule(name="not_null_lastname", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="LastName"),
        ],
        "invoice": [
            DQRowRule(name="not_null_invoiceid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="InvoiceId"),
```

```
            DQRowRule(name="not_null_customerid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="CustomerId"),
            DQRowRule(name="valid_total", criticality="error",
                      check_func=check_funcs.is_in_range, column="Total",
                      check_func_kwargs={"min_limit": 0, "max_limit": 100000}),
            DQRowRule(name="not_null_invoicedate", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="InvoiceDate"),
        ],
        "invoiceline": [
            DQRowRule(name="not_null_invoicelineid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="InvoiceLineId"),
            DQRowRule(name="not_null_invoiceid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="InvoiceId"),
            DQRowRule(name="not_null_trackid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="TrackId"),
            DQRowRule(name="valid_quantity", criticality="error",
                      check_func=check_funcs.is_in_range, column="Quantity",
                      check_func_kwargs={"min_limit": 1, "max_limit": 10000}),
            DQRowRule(name="valid_unitprice", criticality="error",
                      check_func=check_funcs.is_in_range, column="UnitPrice",
                      check_func_kwargs={"min_limit": 0, "max_limit": 10000}),
        ],
        "track": [
            DQRowRule(name="not_null_trackid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="TrackId"),
            DQRowRule(name="not_null_name", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="Name"),
            DQRowRule(name="valid_unitprice", criticality="error",
                      check_func=check_funcs.is_in_range, column="UnitPrice",
                      check_func_kwargs={"min_limit": 0, "max_limit": 10000}),
            DQRowRule(name="valid_milliseconds", criticality="error",
                      check_func=check_funcs.is_in_range, column="Milliseconds",
                      check_func_kwargs={"min_limit": 1, "max_limit": 100000000}),
        ],
        "album": [
            DQRowRule(name="not_null_albumid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="AlbumId"),
            DQRowRule(name="not_null_title", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="Title"),
        ],
        "artist": [
            DQRowRule(name="not_null_artistid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="ArtistId"),
        ],
        "employee": [
            DQRowRule(name="not_null_employeeid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="EmployeeId"),
            DQRowRule(name="not_null_lastname", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="LastName"),
        ],
        "genre": [
            DQRowRule(name="not_null_genreid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="GenreId"),
        ],
        "mediatype": [
            DQRowRule(name="not_null_mediatypeid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="MediaTypeId"),
        ],
        "playlist": [
            DQRowRule(name="not_null_playlistid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="PlaylistId"),
        ],
        "playlisttrack": [
            DQRowRule(name="not_null_playlistid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="PlaylistId"),
            DQRowRule(name="not_null_trackid", criticality="error",
                      check_func=check_funcs.is_not_null_and_not_empty, column="TrackId"),
        ],
    }
    return rules.get(table_name, [])
```

```
%md
## 3. Cleaning Functions (DataFrame Transformations)
```

3. Cleaning Functions (DataFrame Transformations)

```
def clean_string_columns(df):
    """Trim whitespace and normalize empty strings to null for all string columns."""
    for field in df.schema.fields:
        if isinstance(field.dataType, StringType):
            df = df.withColumn(
                field.name,
                when(trim(col(field.name)) == "", None).otherwise(trim(col(field.name)))
            )
    return df

def standardize_country(df):
    if "Country" in df.columns:
        df = df.withColumn("Country", trim(col("Country")))
    return df

def standardize_email(df):
    if "Email" in df.columns:
        df = df.withColumn("Email", lower(trim(col("Email"))))
    return df

def remove_exact_duplicates(df):
    before = df.count()
    df = df.dropDuplicates()
    after = df.count()
    if before != after:
        print(f"    Removed {before - after} exact duplicates")
    return df

def apply_silver_cleaning(df, table_name):
    """Apply all cleaning transformations for Silver layer."""
    df = clean_string_columns(df)
    if table_name == "customer":
        df = standardize_country(df)
        df = standardize_email(df)
    if table_name == "employee":
        df = standardize_email(df)
    df = remove_exact_duplicates(df)
    return df
```

```
%md
## 4. Process Each Bronze Table with DQX Profiling + Validation
```

4. Process Each Bronze Table with DQX Profiling + Validation

```
df_metadata = spark.sql(f"""
    SELECT table_name, primary_key_col
    FROM {catalog}.metadata.pipeline_parent_metadata
    WHERE active_flag = 'Y'
    ORDER BY load_sequence
""")

tables = df_metadata.collect()
silver_results = []
```

› df_metadata:pyspark.sql.connect.dataframe.DataFrame = [table_name: string, primary_key_col: string]

```
for row in tables:
    start_time = datetime.now()
    table_name = row.table_name
    pk_col = row.primary_key_col
```

```
    try:
        # Read Bronze
        bronze_table = f"{catalog}.bronze.{table_name}"
        df_bronze = spark.read.table(bronze_table)
        total_count = df_bronze.count()

        print(f"\n{'='*60}")
        print(f"Processing: {table_name} ({total_count} rows)")
        print(f"{'='*60}")

        # --- CAST DECIMALS TO DOUBLE FOR DQX COMPATIBILITY ---
        df_for_profiling = cast_decimals_to_double(df_bronze)

        # --- DQX PROFILING ---
        print(f"\n  [DQX Profiler] Profiling {table_name}...")
        summary_stats, profiles = profiler.profile(df_for_profiling, options={"sample_fraction": 1.0})
        print(f"  [DQX Profiler] Summary stats:")
        print(yaml.safe_dump(summary_stats, default_flow_style=False))

        # --- DQX GENERATE RULE CANDIDATES ---
        print(f"  [DQX Generator] Generating rule candidates...")
        generated_checks = generator.generate_dq_rules(profiles)
        print(f"  [DQX Generator] Generated {len(generated_checks)} rule candidates:")
        for check in generated_checks[:5]:
            print(f"    {check}")
        if len(generated_checks) > 5:
            print(f"    ... and {len(generated_checks) - 5} more")

        # --- DQX APPLY CHECKS (on cast DataFrame) ---
        dqx_rules = get_dqx_rules(table_name)

        if dqx_rules:
            print(f"\n  [DQX Engine] Applying {len(dqx_rules)} quality checks...")
            valid_df, quarantined_df = dq_engine.apply_checks_and_split(df_for_profiling, dqx_rules)
            passed_count = valid_df.count()
            failed_count = quarantined_df.count()
        else:
            print(f"  [DQX Engine] No rules defined — all records pass")
            valid_df = df_for_profiling
            quarantined_df = spark.createDataFrame([], df_for_profiling.schema)
            passed_count = total_count
            failed_count = 0

        print(f"\n  DQX Result: total={total_count}, passed={passed_count}, quarantined={failed_count}")

        # --- LOG DQ RESULTS TO METADATA ---
        for rule in dqx_rules:
            rule_name = rule.name if rule.name else f"rule_{rule.column}"
            rule_fail_count = 0
            if failed_count > 0:
                try:
                    rule_fail_df = quarantined_df.select(explode("_errors").alias("err")).filter(col("err.name") ==
rule_name)
                    rule_fail_count = rule_fail_df.count()
                except:
                    pass

            spark.sql(f"""
                INSERT INTO {catalog}.metadata.dq_execution_log
                VALUES (
                    '{run_id}', '{table_name}', '{rule_name}',
                    'DQX check_funcs: {rule.column}',
                    {total_count}, {total_count - rule_fail_count}, {rule_fail_count},
                    current_timestamp()
                )
            """)

        # --- QUARANTINE failed records ---
        if failed_count > 0:
            quarantine_table = f"{catalog}.quarantine.{table_name}"
            q_cols_to_drop = [c for c in quarantined_df.columns if c.startswith("_")]
            quarantined_df.drop(*q_cols_to_drop).write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable(quarantine_table)
```

```
            print(f"  Quarantined {failed_count} records → {quarantine_table}")

        # --- SILVER CLEANING on valid records only ---
        print(f"  Applying Silver cleaning transformations...")
        v_cols_to_drop = [c for c in valid_df.columns if c.startswith("_")]
        df_clean = valid_df.drop(*v_cols_to_drop)
        df_clean = apply_silver_cleaning(df_clean, table_name)
        silver_count = df_clean.count()

        # Write to Silver
        silver_table = f"{catalog}.silver.{table_name}"
        df_clean.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable(silver_table)
        print(f"  ✓ Silver written: {silver_count} rows → {silver_table}")

        end_time = datetime.now()
        duration = (end_time - start_time).total_seconds()

        silver_results.append({
            "table_name": table_name,
            "bronze_count": total_count,
            "passed_dq": passed_count,
            "failed_dq": failed_count,
            "silver_count": silver_count,
            "status": "success",
            "duration": duration
        })

        # Log to child metrics
        spark.sql(f"""
            INSERT INTO {catalog}.metadata.pipeline_child_metrics
            VALUES (
                '{run_id}', '{table_name}', 'bronze_to_silver',
                current_timestamp(), 'success',
                {total_count}, {silver_count},
                {passed_count}, {failed_count},
                '{silver_table}', NULL,
                '{start_time}', '{end_time}', {duration},
                current_timestamp()
            )
        """)

    except Exception as e:
        end_time = datetime.now()
        duration = (end_time - start_time).total_seconds()
        error_msg = str(e).replace("'", "''")[:500]
        print(f"  ✗ {table_name}: FAILED — {str(e)[:300]}")

        silver_results.append({
            "table_name": table_name,
            "bronze_count": 0,
            "passed_dq": 0,
            "failed_dq": 0,
            "silver_count": 0,
            "status": "failed",
            "duration": duration
        })

        spark.sql(f"""
            INSERT INTO {catalog}.metadata.pipeline_child_metrics
            VALUES (
                '{run_id}', '{table_name}', 'bronze_to_silver',
                current_timestamp(), 'failed',
                0, 0, 0, 0, '', '{error_msg}',
                '{start_time}', '{end_time}', {duration},
                current_timestamp()
            )
        """)
```

› df_bronze:pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]
› df_clean:pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]
› df_for_profiling:pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]
› quarantined_df:pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 9 more fields]
› valid_df:pyspark.sql.connect.dataframe.DataFrame = [TrackId: integer, Name: string ... 7 more fields]

```
============================================================ Processing: album (347 rows)
============================================================ [DQX Profiler] Profiling album... 22:54:59 INFO
[d.l.d.profiler.generator] ✅ Quality rules generation completed. Generated 5 rules. [DQX Profiler] Summary stats: AlbumId: 25%: 87 50%: 174 75%: 261 count: 347 count_non_null: 347 count_null: 0 max: 347 mean: 174.0 min: 1 stddev: 100.31450543166726 ArtistId: 25%: 58 50%: 112 75%: 180 count: 347 count_non_null: 347 count_null: 0 max: 275 mean: 121.94236311239193 min: 1 stddev: 77.79313102621585 Title: 25%: null 50%: null 75%: null count: 347 count_non_null: 347 count_null: 0 max: Zooropa mean: null min: '...And Justice For All' stddev: null [DQX Generator] Generating rule candidates... [DQX Generator] Generated 5 rule candidates: {'check': {'function': 'is_not_null', 'arguments': {'column': 'AlbumId'}}, 'name': 'AlbumId_is_null', 'criticality': 'error'} {'check': {'function': 'is_in_range', 'arguments': {'column': 'AlbumId', 'min_limit': 1, 'max_limit': 347}}, 'name': 'AlbumId_isnt_in_range', 'criticality': 'error'} {'check': {'function': 'is_not_null', 'arguments': {'column': 'Title'}}, 'name': 'Title_is_null', 'criticality': 'error'} {'check': {'function': 'is_not_null', 'arguments': {'column': 'ArtistId'}}, 'name': 'ArtistId_is_null', 'criticality': 'error'} {'check': {'function': 'is_in_range', 'arguments': {'column': 'ArtistId', 'min_limit': 1, 'max_limit': 275}}, 'name': 'ArtistId_isnt_in_range', 'criticality': 'error'} [DQX Engine] Applying 2 quality checks... DQX Result: total=347, passed=347, quarantined=0 Applying Silver cleaning transformations... ✓ Silver written: 347 rows → workspace.silver.album
============================================================ Processing: artist (275 rows)
============================================================ [DQX Profiler] Profiling artist... 22:55:20 INFO
[d.l.d.profiler.generator] ✅ Quality rules generation completed. Generated 3 rules. [DQX Profiler] Summary stats: ArtistId: 25%: 69 50%: 138 75%: 207 count: 275 count_non_null: 275 count_null: 0 max: 275 mean: 138.0 min: 1 stddev: 79.52986860293433 Name: 25%: null 50%: null 75%: null count: 275 count_non_null: 275 count_null: 0 max: Zeca Pagodinho mean: null min: A Cor Do Som stddev: null [DQX Generator] Generating rule candidates... [DQX Generator] Generated 3 rule candidates: {'check': {'function': 'is_not_null', 'arguments': {'column': 'ArtistId'}}, 'name': 'ArtistId_is_null', 'criticality': 'error'} {'check': {'function': 'is_in_range', 'arguments': {'column': 'ArtistId', 'min_limit': 1, 'max_limit': 275}}, 'name': 'ArtistId_isnt_in_range', 'criticality': 'error'} {'check': {'function': 'is_not_null', 'arguments': {'column': 'Name'}}, 'name': 'Name_is_null',
```

```
%md
## 5. Silver Summary
```

5. Silver Summary

```
import pandas as pd

summary_df = pd.DataFrame(silver_results)
print(f"\n{'='*60}")
print(f"BRONZE → SILVER SUMMARY (Databricks DQX) — Run ID: {run_id}")
print(f"{'='*60}")
print(f"Total tables:      {len(silver_results)}")
print(f"Successful:        {sum(1 for r in silver_results if r['status'] == 'success')}")
print(f"Total quarantined: {sum(r['failed_dq'] for r in silver_results)}")
print(f"{'='*60}")

display(spark.createDataFrame(summary_df))
```

```
› summary_df:pandas.core.frame.DataFrame = [table_name: object, bronze_count: int64 ... 5 more fields]
============================================================ BRONZE → SILVER SUMMARY (Databricks DQX) — Run ID: 2d0ac997 ============================================================ Total tables: 11 Successful: 11 Total quarantined: 0
============================================================
```

Table

```
%md
## 6. View DQX Execution Log
```

6. View DQX Execution Log

```
display(spark.sql(f"""
    SELECT table_name, rule_name, rule_description, total_processed, passed_count, failed_count
    FROM {catalog}.metadata.dq_execution_log
    WHERE run_id = '{run_id}'
    ORDER BY table_name, rule_name
"""))
```

Table

```
%md
## 7. View Quarantine (if any)
```

7. View Quarantine (if any)

```
quarantine_tables = [r["table_name"] for r in silver_results if r["failed_dq"] > 0]
if quarantine_tables:
    for qt in quarantine_tables:
        print(f"\nQuarantine: {catalog}.quarantine.{qt}")
        display(spark.sql(f"SELECT * FROM {catalog}.quarantine.{qt} LIMIT 5"))
else:
    print("No records quarantined — all data passed DQX validation checks")
```

No records quarantined — all data passed DQX validation checks

```
dbutils.jobs.taskValues.set(key="run_id", value=run_id)
```